

形式语言与编译 2024 考试题（回忆版）参考答案

来源：2024 形编考试题回忆版.docx、24 年考试题回忆版草稿.md

说明：本份只解可确定的题

涉及章节：本卷为回忆版，部分题目缺少原始数据或存在笔误，无法严谨复原。以下只给**题面与数据完整、可确定作答**的大题；其余题目仅列出无法作答的原因，避免给出臆测答案。

可确定作答：

- 第五题语法分析：文法与句子完整。
- 第六题 itemDFA 与冲突：增广文法完整。
- 第七题语义分析：程序段完整。
- 第八题运行时栈快照：回忆版程序段有笔误，下面已修正成自洽版本后作答（修正说明见该节）。

暂不作答（数据不全）：

- 选择题：选项不全，只有题意描述。
- 第二题 NFA $\rightarrow$ 最小 DFA $\rightarrow$ RE：未给出具体 NFA 迁移表。
- 第三题词法分析：未给出具体词法 DFA。
- 第四题文法修剪：三小问的具体文法未在回忆版中给全。

五、语法分析（10 分）

涉及章节：Slides：第五章 2026.pdf、第八章-2026new.pdf；笔记：第 5 章“推导、语法树、短语、直接短语、句柄、二义文法”，第 8 章“规范归约”。

文法（约定 i 表示一个数字记号，如 2、3、4）：

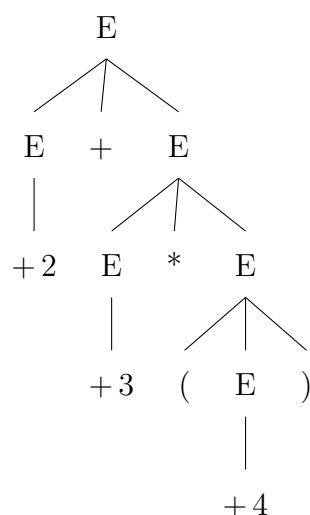
$$E \rightarrow E * E \mid E + E \mid +i \mid (E).$$

句子： $+2 + +3 * (+4)$ ，即 $+2 + +3 * (+4)$ 。

5.1 规范归约；直接短语与句柄

先确定语法树（题目已给，这里按文法复原其结构）。按通常的优先级（ $*$ 高于 $+$ ）与左结合，句子括号化为：

$$((+2) + (+3 * (+4))).$$



规范归约是最右推导的逆过程：每一步把当前句型中**最左的直接短语（句柄）**归约为对应非终结符。逐步如下（ $\leftarrow$ 表示一次归约）：

步	当前句型	归约所用产生式（句柄加粗）
0	+2 + +3 * (+4)	句柄 +2 ，用 $E \rightarrow +i$
1	E + +3 * (+4)	句柄 +3 ，用 $E \rightarrow +i$
2	E + E * (+4)	句柄 +4 ，用 $E \rightarrow +i$
3	E + E * (E)	句柄 (E)，用 $E \rightarrow (E)$
4	E + E * E	句柄 E * E ，用 $E \rightarrow E * E$
5	E + E	句柄 E + E ，用 $E \rightarrow E + E$
6	E	归约完成，得到开始符号

直接短语：语法树中“父结点一步推出、其子结点恰为某产生式右部”的子串。本句的直接短语为

+2, +3, +4.

$((E)$ 、 $E * E$ 、 $E + E$ 是后续步骤的直接短语，但对最初的句子而言，最底层一步推出的是三个 $+i$ 。）

句柄：规范归约中最先被归约的直接短语，即最左的那个：

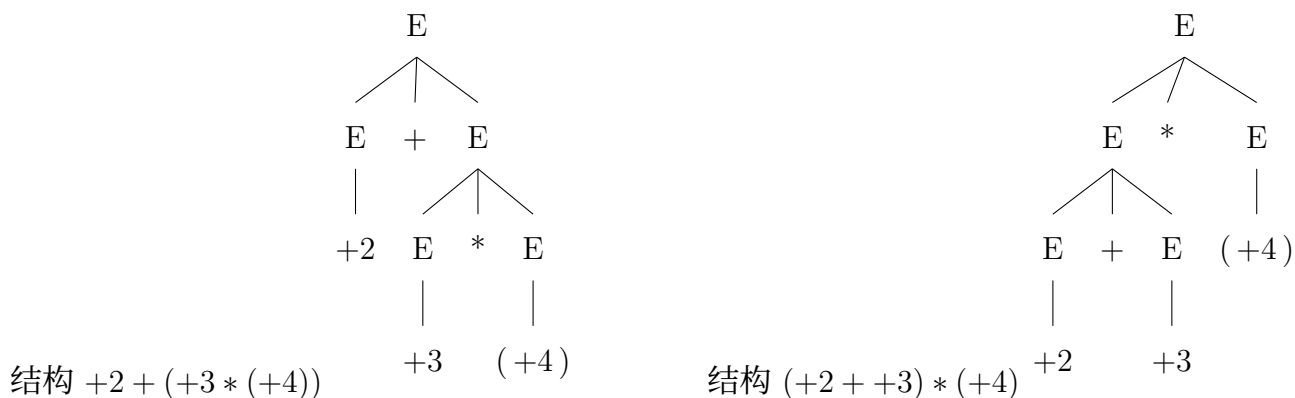
句柄 = +2.

5.2 是否二义

该文法是二义文法。文法对 $*$ 、 $+$ 既没规定优先级也没规定结合性，所以同一句子可有不同语法树。以子串 $+2 + +3$ 与后面的 $*(+4)$ 为例， $+3 * (+4)$ 这段就有两种结合：

$$+2 + (+3 * (+4)) \quad \text{与} \quad (+2 + +3) * (+4).$$

两种括号化对应两棵不同的语法树（即两个不同的最右推导），它们生成同一个句子，故文法二义。



六、itemDFA 与冲突 (10 分)

涉及章节: Slides: 第八章-2026new.pdf; 笔记: 第 8 章“LR(0) 项目、活前缀 DFA、移进-归约冲突、SLR(1)”。

增广文法:

$$S' \rightarrow E, \quad E \rightarrow E + E \mid +E \mid +i.$$

6.1 LR(0) 项目集 (itemDFA 的状态)

$$I_0 = \{S' \rightarrow \cdot E, E \rightarrow \cdot E + E, E \rightarrow \cdot + E, E \rightarrow \cdot + i\},$$

$$I_1 = \{S' \rightarrow E \cdot, E \rightarrow E \cdot + E\},$$

$$I_2 = \{E \rightarrow + \cdot E, E \rightarrow + \cdot i, E \rightarrow \cdot E + E, E \rightarrow \cdot + E, E \rightarrow \cdot + i\},$$

$$I_3 = \{E \rightarrow E + \cdot E, E \rightarrow \cdot E + E, E \rightarrow \cdot + E, E \rightarrow \cdot + i\},$$

$$I_4 = \{E \rightarrow +E \cdot, E \rightarrow E \cdot + E\},$$

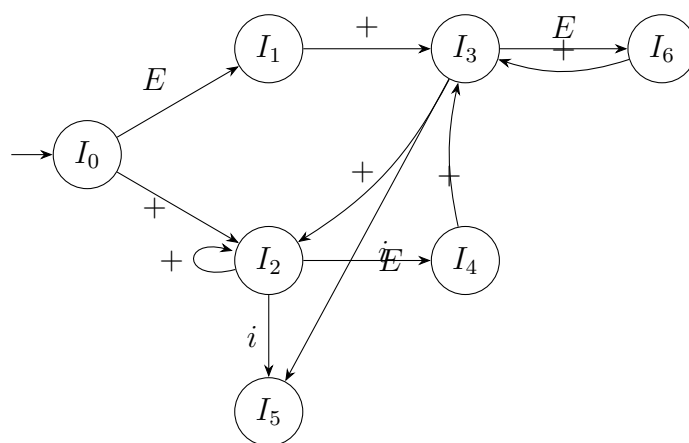
$$I_5 = \{E \rightarrow +i \cdot\},$$

$$I_6 = \{E \rightarrow E + E \cdot, E \rightarrow E \cdot + E\}.$$

6.2 goto 转移表 (itemDFA)

状态	E	$+$	i
I_0	I_1	I_2	
I_1		I_3	
I_2	I_4	I_2	I_5
I_3	I_6	I_2	I_5
I_4		I_3	
I_5			
I_6		I_3	

对应的活前缀 DFA:



6.3 冲突分析

先求 FOLLOW 集。 E 是开始符号 (经 $S' \rightarrow E$), 故 $\# \in FOLLOW(E)$; 又由 $E \rightarrow E + E$ 知 $+\in FOLLOW(E)$ 。所以

$$FOLLOW(E) = \{+, \#\}.$$

存在两个移进-归约冲突, 且 SLR(1) 也无法消解:

- 状态 I_4 : 含归约项目 $E \rightarrow +E\cdot$ 与移进项目 $E \rightarrow E\cdot+E$ 。归约要求的输入是 $FOLLOW(E) = \{+, \#\}$, 其中 $+$ 同时又是移进符号, 因此在 $+$ 上移进-归约冲突。
- 状态 I_6 : 含归约项目 $E \rightarrow E + E\cdot$ 与移进项目 $E \rightarrow E\cdot + E$ 。同理在 $+$ 上移进-归约冲突。

因为归约用的 $FOLLOW(E)$ 里含有移进符号 $+$, SLR(1) 不能区分, 所以这是真正的二义文法引起的冲突 (与第五题 “ $E \rightarrow E + E$ 无结合性” 一致)。

6.4 任意消除其中一个

按题目要求消除一个即可。取状态 I_6 的冲突, 规定 + 左结合:

在 I_6 遇到输入 + 时, 选择按 $E \rightarrow E + E$ 归约 (reduce), 不移进。

理由: 左结合意味着先把左边已凑齐的 $E + E$ 规约掉, 再处理后续的 +。这样 I_6 在 + 上只保留归约动作, 移进-归约冲突被消除。(若要消 I_4 , 同理可令一元 + 优先级更高、在 + 上选择归约 $E \rightarrow +E$ 。)

七、语义分析 (15 分)

涉及章节: Slides: 第九章 2026.pdf、第十章 2026.pdf; 笔记: 第 9 章“符号表、声明语义分析、函数声明、综合属性与继承属性”, 第 10 章“三地址码、四元式、短路求值、数组访问、函数调用”。

程序段:

```
int a[2, 3];
void f(int b(), int k){
    if(k > 0 && k < 3) a[k-1,k] = b(k, 1)
    else print 0;
}
```

7.1 对 f 函数声明行做语义分析 (符号表)

涉及两张表: 全局表, 以及 f 的局部/参数表。

全局符号表 (int a[2,3]; 与 f 同层):

name	kind	type / 说明	width	offset
a	array	元素 int, dims=[2,3], 行优先	$2 \times 3 \times 4 = 24$	0
f	function	returnType=void, paramNum=2, paramList=[b,k], 指向 f 的局部表	—	24

a 的宽度: 二维数组 int[2,3], 元素 int 占 4, 共 $2 \times 3 = 6$ 个, 故 $width = 24$; f 紧随其后, 偏移从 24 起。

分析 void f(int b(), int k) 的步骤:

- (1) 登记函数名: 在全局表查 f 是否重定义 (无同名, 通过), 登记 f, kind=function, returnType=void。
- (2) 新建 f 的局部表: newtab(), parent 指向全局表, 层次加深一层; 全局表中 f 的 entry 指向该子表。

(3) 逐个登记形参（从左到右）：

- `int b()`：b 是**函数型形参**（参数本身是函数，返回 `int`、无参数）。它不是普通变量，运行时要携带**入口地址**和**环境信息**（访问链/静态链），故 `kind=funcParam`。
- `int k`：普通整型形参，`kind=param`，`type=int`，`width=4`。

(4) 回填函数信息：更新 f 的 `paramNum=2`、`paramList=[b,k]`、参数区总宽度。

f 的局部/参数符号表：

name	kind	type / 说明	width	offset
b	funcParam	函数形参， <code>returnType=int</code> ，无参数；存“入口地址 + 环境”	按机器（如 8）	0
k	param	<code>int</code> 形参	4	接 b 之后

关键考点：b 是“函数当参数”，不能只当成一个 `int`。普通变量只占一个数据宽度，而函数形参要记**入口地址**（去哪执行）和**环境信息**（按什么作用域取非局部名）。

自然语言版：声明了一个全局函数 f，返回 `void`，两个参数；第一个参数 b 是返回 `int` 的函数（作为参数传入，运行时带入口地址和环境），第二个参数 k 是 `int`。在全局表登记 f 并建立其作用域，把 b、k 登记到参数表，最后回填参数个数、参数类型表和返回类型。

7.2 对 if 语句写三地址码 / 四元式

控制结构：`k>0 && k<3` 用短路(`k>0` 假就直接跳 `else`)；`then` 分支是数组赋值 `a[k-1,k]=b(k,1)`，结束后跳过 `else`。

带标签三地址码（最直观）：

```
        if k > 0 goto L1      // k>0 真才判断 k<3 (&& 短路)
        goto Lelse           // k>0 假，整体假
L1:      if k < 3 goto Lthen
        goto Lelse
Lthen:   t1 = k - 1           // 下标 k-1
        t2 = t1 * 3           // 行优先 (i*n+j), n=3, i=k-1, j=k
        t3 = t2 + k
        param k               // b(k,1): 从左到右传参
        param 1
        t4 = call b, 2        // 调用 b, 2 个参数，返回值入 t4
        a[t3] = t4            // 写回数组元素 a[k-1,k]
        goto Lend            // then 结束，跳过 else
Lelse:   print 0
Lend:
```

要点：

- **&& 短路**: $k > 0$ 真链进入 $k < 3$ 判断, 假链直接到 **Lelse**; 两条件都真才进 **Lthen**。
- **数组下标 $a[k-1, k]$** : 声明 $a[2, 3]$, 行优先偏移 $(i \cdot n + j)$, 这里 $i = k - 1$ 、 $j = k$ 、 $n = 3$, 算出的是**元素编号**。若要字节地址, 再乘宽度 4: $t3 = t3 * 4$, 写成 $M[\text{base}(a) + t3] = t4$ 。
- **函数调用 $b(k, 1)$** : 先 **param** 传参 (无约定时从左到右), 再 **call b, 2**; **b** 是函数型形参, 运行时按符号表里记的入口地址和环境调用。

四元式 $((op, arg1, arg2, result))$, 跳转的 **result** 为目标四元式编号, 自 100 起):

编号	op	arg1	arg2	result
100	$j >$	k	0	102
101	j	—	—	112
102	$j <$	k	3	104
103	j	—	—	112
104	—	k	1	t1
105	*	t1	3	t2
106	+	t2	k	t3
107	param	k	—	—
108	param	1	—	—
109	call	b	2	t4
110	$[] =$	t3	t4	a
111	j	—	—	113
112	print	0	—	—
113	...			(后续语句)

说明: $j >/j <$ 表示“条件成立跳转到 **result** 指定编号”, j 是无条件跳转, $[] =$ 表示数组元素赋值 $a[t3] = t4$ 。100/102 的真出口经反填指向各自的下一段; 101/103 的假出口反填到 **else** 入口 112。第 110 条之后用 111 无条件跳过 **else** (跳到 113)。具体标号衔接可按课件标号方式微调。

说明: 本题与笔记“第 9 章·例题: 2024 语义分析”一致, 可对照复习。

八、运行时存储空间组织 (10 分)

涉及章节: Slides: 第十一章 2026.pdf; 笔记: 第 11 章“活动树、栈帧/活动记录、控制链与访问链、参数传递、栈快照通用步骤”。

8.1 题面修正

回忆版程序段有笔误，按要求统一并修正成语义自治的版本（修正点：`raw`→`row`；`soo` 作为“函数型形参”，被调用时统一带一个实参 `soo(x)`；`bar` 递归时把当前的 `soo` 继续往下传）：

```
int foo(int y){
    int z;
    void bar(int x, int soo()){
        if(x > 3) bar(x/3, soo);    // 把函数 soo 继续往下传
        else z = soo(x);           // 调用函数型形参 soo
        print z;
    }
    int row(int x){
        y = x + 5;
        return y;                  // <== 求执行到这里时的栈快照
    }
    bar(y, row);                   // 把函数 row 作为参数传给 bar
}
foo(6);
```

词法嵌套关系：`bar` 和 `row` 都直接声明在 `foo` 内，所以两者的访问链都指向 `foo` 的活动记录。

8.2 调用链推演

- (1) `foo(6)`: `y=6`。执行 `bar(y, row)`，即 `bar(6, row)`，把函数 `row`（连同它的环境 `foo`）作为参数 `soo` 传入。
- (2) `bar(6, row)`: `x=6>3` 成立，递归 `bar(6/3, soo)`，即 `bar(2, row)`，`soo` 继续是 `row`。
- (3) `bar(2, row)`: `x=2>3` 不成立，走 `else`，执行 `z = soo(x)`，即 `z = row(2)`。
- (4) `row(2)`: `y = 2 + 5 = 7`，执行到 **`return y`**。此刻栈顶是 `row` 的活动记录。

因此运行到 `return y` 时，栈上由底到顶依次有 4 个活动记录：

`foo(6) → bar(6,row) → bar(2,row) → row(2)`。

此刻的值：`y` 已被 `row` 改成 7；`z` 还未赋值（`z=row(2)` 的右边正在求值，赋值尚未完成），故 `z` 仍为未定义。

8.3 栈快照

按题图字段顺序（形参 / 访问链 / 控制链 / 返回地址 / 变量）自底向上填写。`y`、`z` 是 `foo` 的数据，登记在 `foo` 的活动记录里；函数型实参 `soo` 携带一对值“代码入口 + 环境”，环境即声明它的 `foo` 活动记录。用记号 `AR(p)` 表示过程 `p` 本次活动的活动记录基址。

活动记录	单元 (字段)	内容 / 说明
foo(6)	形参 y	7 (初值 6, 已被 row 改写为 $2 + 5$)
	控制链	指向调用者 (主程序/#)
	局部变量 z	未定义 ($z = \text{row}(2)$ 尚未赋值)
bar(6, row) 第 1 次	形参 x	6
	形参 soo	(row@ 入口, AR(foo)), 即 row 的代码入口 + 环境 foo
	访问链	指向 AR(foo) (bar 声明在 foo 内)
	控制链	指向 AR(foo) (调用者是 foo)
	返回地址	foo 中 bar(y, row) 之后
bar(2, row) 第 2 次 (递归)	形参 x	2 ($= 6/3$)
	形参 soo	(row@ 入口, AR(foo)), 由上一层 bar 原样传下
	访问链	指向 AR(foo) (仍按声明环境, 不是调用者)
	控制链	指向第 1 次 bar 的活动记录 (调用者)
	返回地址	bar 中 if 的 then 分支 bar(x/3, soo) 之后
row(2)	形参 x	2
	访问链	指向 AR(foo) (row 声明在 foo 内; 由 soo 携带的环境得到)
	控制链	指向第 2 次 bar 的活动记录 (调用者)
	返回地址	bar 中 $z = \text{soo}(x)$ 的赋值处

两条链的要点 (本题最易错处):

- **访问链按词法嵌套** (声明位置) 设置: bar、row 都声明在 foo 内, 所以三个活动记录的访问链**都指向 foo**, 与“谁调用谁”无关。row 被当作 soo 间接调用, 它的访问链也来自传入的环境 AR(foo)。
- **控制链按调用顺序**设置: row 的控制链指向第 2 次 bar, 第 2 次 bar 的控制链指向第 1 次 bar, 第 1 次 bar 的控制链指向 foo。
- **函数型形参 soo**: 不是普通整数, 存的是一对“代码入口 + 环境”。正因为带了环境 AR(foo), row 里的非局部名 y 才能正确地找到 foo 的 y 并改成 7。

说明: 回忆版未给出具体地址数值与帧指针标法, 本表用符号 AR(p) 表达链的指向关系; 若老师要求填具体地址, 可仿照 exam_17 第 7 题, 从高地址往低地址给每个单元编号即可, 链值改成对应单元地址。